Task 1: Import Required Libraries

```
[1]
2s
import numpy as py
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
import geopandas as gpd
import folium
```

Task 2: Load Natural Earth Dataset

```
...
          featurecla  scalerank  LABELRANK                   SOVEREIGNT SOV_A3  \
0  Admin-0 country           1          6                         Fiji    FJI
1  Admin-0 country           1          3  United Republic of Tanzania    TZA
2  Admin-0 country           1          7                Western Sahara    SAH
3  Admin-0 country           1          2                        Canada    CAN
4  Admin-0 country           1          2     United States of America    US1

   ADM0_DIF  LEVEL               TYPE TLC                        ADMIN  ...  \
0         0      2  Sovereign country   1                         Fiji  ...
1         0      2  Sovereign country   1  United Republic of Tanzania  ...
2         0      2      Indeterminate   1                Western Sahara  ...
3         0      2  Sovereign country   1                        Canada  ...
4         1      2            Country   1     United States of America  ...

       FCLASS_TR     FCLASS_ID     FCLASS_PL FCLASS_GR  FCLASS_IT  \
0           None          None          None      None       None
1           None          None          None      None       None
2  Unrecognized  Unrecognized  Unrecognized      None       None
3           None          None          None      None       None
4           None          None          None      None       None

       FCLASS_NL FCLASS_SE  FCLASS_BD FCLASS_UA  \
0           None      None       None      None
1           None      None       None      None
2  Unrecognized      None       None      None
3           None      None       None      None
4           None      None       None      None

                                            geometry
0  MULTIPOLYGON (((180 -16.06713, 180 -16.55522, ...
1  POLYGON ((33.90371 -0.95, 34.07262 -1.05982, 3...
2  POLYGON ((-8.66559 27.65643, -8.66512 27.58948...
3  MULTIPOLYGON (((-122.84 49, -122.97421 49.0025...
4  MULTIPOLYGON (((-122.84 49, -120 49, -117.0312...
```

1. 169 Columns

2. Polygon and Multi-Polygon

3. The column represents the spatial boundaries of each country, stored as POLYGON or MULTIPOLYGON objects in coordinate form (longitude, latitude).

Task 3: Check the Coordinate Reference System

```
print("Current CRS:", world.crs)

...  Current CRS: EPSG:4326
```

1. EPSG:4326 is the code for the WGS84 geographic coordinate system, which uses latitude and longitude in degrees to represent locations on Earth. It's the most widely used global standard for GPS and mapping.

2. A Coordinate Reference System (CRS) is crucial in spatial analysis because it defines how geographic data relates to real-world locations. Without a CRS, coordinates are just numbers with no spatial meaning.

Task 4: Convert to Metric CRS for Area Calculation

```
#Reproject to Metric CRS
world_projected = world.to_crs(epsg=3857)

#Compute Area
world_projected["area_m2"] = world_projected.geometry.area

print(world_projected[["ADMIN", "area_m2"]].head())
print("\nArea Reprojection Completed")
```

```
                          ADMIN       area_m2
0                          Fiji  2.128334e+10
1  United Republic of Tanzania  9.522552e+11
2                Western Sahara  1.171023e+11
3                        Canada  5.216648e+13
4      United States of America  2.186228e+13

Area Reprojection Completed
```

1. You can't compute area accurately with EPSG:4326 because it's a geographic CRS. Coordinates are in degrees of latitude and longitude, not in linear units. Degrees don't correspond to consistent distances across the globe (they vary with latitude), so area calculations would be distorted.

2. EPSG:3857 (Web Mercator) is a projected CRS based on the WGS84 datum. Its units are meters, which makes it suitable for calculating areas, distances, and performing metric-based spatial analysis.

Task 5: Extract Centroid Coordinates

```
[10]
✓ 0s
#Extract Centroid

world_projected["centroid"] = world_projected.geometry.centroid
world_projected["x"] = world_projected.centroid.x
world_projected["y"] = world_projected.centroid.y

print(world_projected[["ADMIN", "x", "y"]].head())
```

```
                          ADMIN             x             y
0                          Fiji  1.824878e+07 -1.958098e+06
1  United Republic of Tanzania  3.869296e+06 -7.003071e+05
2                Western Sahara -1.348403e+06  2.794163e+06
3                        Canada -1.079779e+07  1.044422e+07
4      United States of America -1.329713e+07  6.667416e+06
```

1. A centroid is the geometric center of a shape, representing the average position of all its points. For polygons, it's the "balance point" of the geometry.

2. Centroid coordinates (x, y) can be used as representative points of regions in clustering algorithms. They allow grouping countries or areas by spatial proximity, helping identify regional patterns or clusters.

3. We separate x (longitude/easting) and y (latitude/northing) because clustering and machine learning models require numeric feature columns. Having them split makes it easier to compute distances, run algorithms, and visualize results.

Reflection

What difficulties did you encounter?

I struggled with understanding why areas couldn't be calculated directly in EPSG:4326, and I had to learn the difference between geographic CRS (degrees) and projected CRS (meters).

What did you learn about spatial data?

I learned that spatial data isn't just coordinates it depends heavily on the CRS. Reprojecting to the right system makes calculations like area, distance, and clustering accurate and meaningful.

Colab Notebook Link:
https://colab.research.google.com/drive/1OT1NRuqM6JFsi5MqtkjRjxqp4TfjF6uX?usp=sharing